Hexagonal Network Structures

Daniel Lopez

August 2025

Abstract

We propose and establish baselines for a novel, albeit impractical neural network topology that departs from traditional Cartesian grid layouts of neurons and connections and employs a hexagonal lattice enabling neighboring networks with overlapping weights, Hexagonal Neural Networks. Inspired by extreme behavior compartmentalization behavior disorders observed in the general populace, this network structure focuses on the influence of directional training on overlapping weights. Early baselines reveals new learning behaviors through loss and accuracy iterations and consistent improvement from single direction propagation which has widespread usage in modern day commercial deep learning structures.

1 Introduction

Multilayer perceptrons are traditionally structured as a unidirectional chain of fully-connected layers, where each layer only connects to the next in sequence. While this base model has had several iterations that are found in modern day powerful AI systems, most commercial systems stick to this linear direction of deep learning, given how well this can scale. Significant progress has been made on this front as geometric deep learning has explored architectures that are robust to different transformations of data. For example, they are invariant to affine transformations such as translation, rotation, scaling.

However, the expressive potential of neural computation may not be limited to such linear structures.

Arbitrary Neural Networks, for example, have sparse research surrounding it since they are impractical for scaling in terms of memory, unstructured layers, and time for experimentation. More heavily researched architectures, however, include group equivariant CNNs, spherical CNNs, and message passing neural networks in efforts to incorporate symmetry into neural processing. Specifically, the filters that are used in convolution layers are rotated for angular invariance of features in the context of image recognition. While this symmetry tends to help the network stay robust in equivariance of the system, it still is a single network in a linearly trained direction.

Borrowing from this tradition of symmetry, but allowing multidirectional tiling and directionality, consider a Hexagonally constructed neural network. We can preserved bipartite connections between layers to retain the fully connected nature of MLPs, but we can gain rotation redundancy of hexagonal tilings, and expose two more networks with shared weights. Here, we introduce Hexagonal Neural Networks (HexNNs) that may generalize MLP into a rotationally symmetric, multi-directional structure.

HexNNs replace the standard 1D layer stack with an initial 2D topological arrangement made to resemble regular hexagons. Given a size parameter, n, the network has equal n length input and output layers, with hidden layers symmetrically increasing and decreasing to $2n - 1$. Crucially, each node in layer i is connected to all nodes in layer $i + 1$, but unlike MLPs, this bipartite adjacency is true for all three cardinal directions of a hexagon. This allows for rotation-based reinterpretation of the network's directionality.

This design enables six distinct orientations of the same network, each corresponding to a 60° rotation of the hexagonal structure. We define each orienta-

tion as a subset of the network, H_i, and note that forward propagation of any H_i is equivalent to backwards propagation of its corresponding opposite network, $H_{(i+3) \bmod 6}$.

1.1 Open Questions

This architectural symmetry leads to several research questions that can uniquely be defined by this structure, but primarily: *if we train a hexagonal neural network, what impact does that have on the performance of the networks in other directions?*

2 Background

2.1 Human Inspiration

Neural Networks are a leading heuristic for general function approximation given how well they are able to lend themselves to arbitrarily complex non linear correlations between high dimensional spaces. Their structure is heavily inspired by similar structures in cerebral connections. Human cognition and artificial intelligence thus are known to face common challenges, such as managing knowledge across diverse tasks without losing prior knowledge gain. Humans and other cognitively advanced organisms generally use **integration** to share knowledge between tasks through social structures. They share tribal knowledge by teaching new cohort members by showing and doing. Modern AI systems use the opposite approach, **compartmentalization**, by focusing the training and testing of such systems into specific "contexts" to avoid interference of clashing data sets. Even omni-model AIs use a small ecosystem of these focused AIs that the orchestrator merely summarizes back to the user, giving the illusion of a single model. [1] This is largely done to prevent a phenomenon known as "catastrophic forgetting" which is most likely to happen when an AI is presented with a continuous stream of new information.

Compartmentalization can still occur in humans, however. For example, an extreme example thereof can be disassociate identity disorder (DID), where memories are split between identities.

The robustness of neural networks comes from the breadth of their intra layer connections. In a typical MLP, there are several

3 Implementation

3.1 Hexagonal Neural Network

A hexagonal neural network is a multilayer perceptron network whose layer sizes increase and then decrease symmetrically, forming a hexagon-like outline if it were to be drawn in 2D.

As expected from feed-forward networks, all neurons in any given layer are fully connected to all neurons in the adjacent layers, but this is not just true in one direction. Due to the hexagonal nature of the framework, there are six distinct directions in which one could perform traditional forward and backward neural network propagation. This renders the structure to contain a unique corresponding arrangement of connections and shared weights given the direction. These shared weights may potentially influence the internal network's representation of data in their high-dimensional embedding from one propagation direction to another. In other words, training the network with training data from one of these directions could potentially help training the same network with the same data from another angle to escape local optima encountered during gradient descent or other optimization methods.

Below, we present a mathematical generalization of this architecture, show adjacency matrices that will hopefully showcase the patterns that emerge as n grows, optimize memory allocation for arbitrarily large n values, and see the effects of loss and accuracy that training in one direction can have on other directions.

3.2 Hexagonal Connectivity and Spatial Directions

A key property of this network is its hexagonal symmetry. Unlike a traditional MLP with strict linearity

[1] FIND REF

from left to right, the hexagonal network, H, has connections that can be interpreted in three unique spatial directions. Although we do have six unique networks, we call these three directions, since forward propagation of one network, say H_1, can be interpreted as the backward propagation of the network 3 rotations away; in this example, H_4. To be explicit, if we visualize the network with the layers centered and stacked vertically (i.e. with the smallest layers at the top and bottom, widest in the middle), then we have the following directions:

- **Vertical Direction:** Connections that go straight "down" from one layer to the next. For simplicity, straight-down propagation can be represented as H_1 and its backward propagation as H_4. Note that although H_1 and H_4 are the same neural network, they are mirror opposites of forward and back propagation.
- **Down-Right Direction:** Connections that start from one "hexagonal rotation" clockwise from the vertical H_1. These are H_2 and its inverse H_5
- **Down-Left Direction:** Connections that start from one hexagonal rotation counter clockwise from H_1. These are H_3 and H_6.

The layer sizes of N follow an explicit pattern:

$$L = [n, n+1, ...2n-1, ..., n+1, n]$$

This gives the total number of layers $N_L = 2n-1$. The total number of nodes in a network, however, is:

$$T = \sum_{k=0}^{2n-2} (n + \min(k, 2n-2-k))$$

For the next few mathematical definitions, let the **Vertical Direction** orientation of the network, H_1, be the assumed direction. Furthermore, let's assume the numbering of the network to start from the top left index, traverse left to right, and then top to bottom. Let:

- $A^i \in \{0,1\}^{T \times T}$ be the binary adjacency matrix for arbitrary direction H_i
- For an neuron, n, at $i \in [0, T-1]$, or n_i, let layer(i) map it to its layer number $l \in [0, N_L - 1]$
- S_l be the set of indices in layer l

Then, the H_1, the only non-zero entries in A are defined as:

$$A_{ij} = \begin{cases} 1 & i \in S_l, j \in S_{l+1}, \text{ for some } l \in [0, N_L - 2] \\ 0 & \text{otherwise} \end{cases}$$

A symptom of this definition for H_1 is a block structure that gets created inside A^1. Each block $(l, l+1)$ is a dense matrix of ones with size $|S_l| \times |S_{l+1}|$ Thus, A^1 can be visualized as:

$$A^1 = \begin{bmatrix} 0 & B_0 & 0 & \dots & 0 \\ 0 & 0 & B_1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & B_{N_L-2} \\ 0 & 0 & \dots & 0 & 0 \end{bmatrix}$$

where every $B_k \in \{0,1\}^{|S_k| \times |S_{k+1}|}$ is a matrix of all 1s. We can see this pattern hold as n grows for H_1 in 1

3.2.1 Rotations

As previously noted, properties such as layer sizes and layer weights for H_i will be the reverse for the geometric opposite network, $H_{(i+3) \bmod 6}$, and therefore, adjacency matrices, weight matrices, and other relevant matrices can be reused.

3.3 Traditional Neural Network Operations

The connections between nodes in any neural network serve as weights, and thus the adjacency matrix A^i can be $W^i \in \mathbb{R}^{T \times T}$ where all values that are not zero as previously defined are now a real number.

Thus, for forward propagation, you can just iteratively compute:

$$x^{t+1} = f(W^T \times x^t)$$

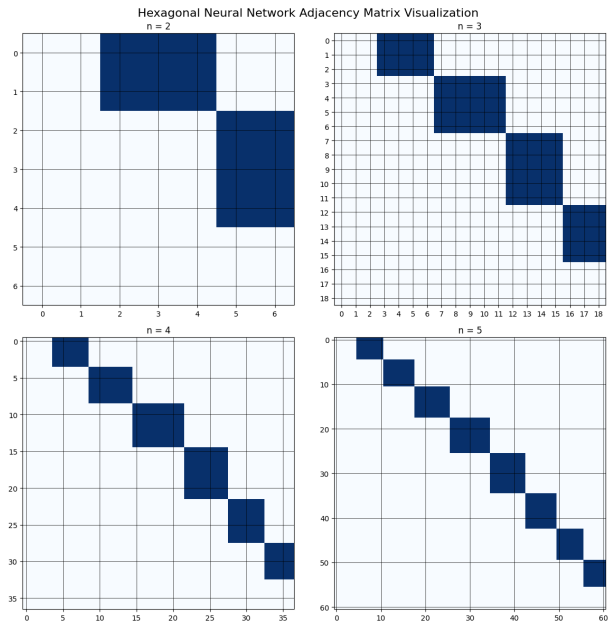

Figure 1: Adjacency Matrix showcasing the Block structure for H_1

where f is a typical activation function for neural networks, and x is the padded form of the input vector. This padding brings x from n dimensions to T dimensions for ease of multiplication with W.

3.3.1 Example Flow with $n = 2$

In a network with $n = 2$,

- The layer sizes are $[2, 3, 2]$
- The total number of nodes is $T = 7$
- The Weight and Adjacency matrices $A, W \in \mathbb{R}^{7\times 7}$
- The padded input vector x is

$$x = \begin{bmatrix} x_0 \\ x_1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

- Computation is simply:

3.4 Network Variance

If we interpret six co-dependent multi layer perceptrons as an ensemble with parameter tying, here are a few variants that we could try finding baselines within the guidance of ensemble based learning.

We will establish some baseline models, and their expected outputs. Then we will try five variants of these baselines that are based on exploits and manipulations that we can do exclusively because of hexagonal nature of the graph. To these five, we will then be testing some orthogonal knobs. We will establish what are the independent and dependent and testing variables that we will be fine tuning and sweeping through to test.

First, let's establish some baselines.

3.4.1 Baselines

A single model, without sharing, standard training Deep ensemble, six separate networks, no sharing, independent seeds. Snapshot ensemble / SWA: 1 network, multi snapshot averaging MC-Dropout: 1 net, inference-time dropout

3.4.2 Variants 1 - 5: Hex Ensemble Variants

Variant 1: Hard Rotation Sharing: One backbone tensor bank, each member 'applies a hex rotation' to filters adjacency before its forward. Only small member-specific heads (and optional tiny offsets) are unique. Combines group equivariance with ensembling

Variant 2: Soft sharing via low-rank delta (LoRA-style) Shared backbone θ; each member has $\Delta_i = A_i B_i$ (rank-r) on selected layers. Control r to

trade diversity vs. parameters Optimal: permember BN/Layer Norm

Variant 3: Masked weight reuse Shared tensor bank W; each member uses a binary/ternary mask M_i (learned or fixed) such that $W \circ M_i$ Encourages orthogonal subspaces with negligble memory

Variant 4: Shared backbone, member heads only Classic multi head: 100 percent sahred feature extractor; six heads (Acts as lower bound on diversity)

Variant 5: Optimizer state-sharing vs isolatioin Same weights but either (i) one optimizer for all members (cotraining) or (ii) per member optimizers on their delta/heads only.

4 Result

4.1 Base Case

Baselines for H_0 have been created where loss function, activation functions, and n are variable, and we can measure their performance with regression metrics such as scalar loss (according to the respective loss function), accuracy (root mean square deviation, as is standard for regression based data sets) and R^2 (the coefficient of determination).

4.2 Simulated Data Setup

I did things here (identity dataset, linear dataset, chat gpt outputted)

4.3 Real Data Sets

I did things with real data here